

Lesson 15 Handout – Semi-gradient n -step SARSA with Linear VFA

Reinforcement Learning

DSOR 646

This handout shows a Python implementation of the Semi-gradient n -step SARSA algorithm, an on-policy temporal difference (TD) control solution procedure. The pseudocode for the algorithm is displayed on page 247 of the course textbook. A linear value function approximation (VFA) scheme using a tile coding mechanism with binary features is applied to represent the value of state-action pair. The algorithm determines a policy and value function for the *MountainCar-v0* Farama Foundation Gymnasium Environment. The provided code generates a graph displaying the *learning curve* and other important algorithm performance statistics. See Figure 1. The *tiles3.py* Python file for the tile coding – as given by Sutton – is provided in the Lesson 15 entry on Canvas. Further information about the tile coding implementation is provided by Sutton here: <http://incompleteideas.net/tiles/tiles3.html>.

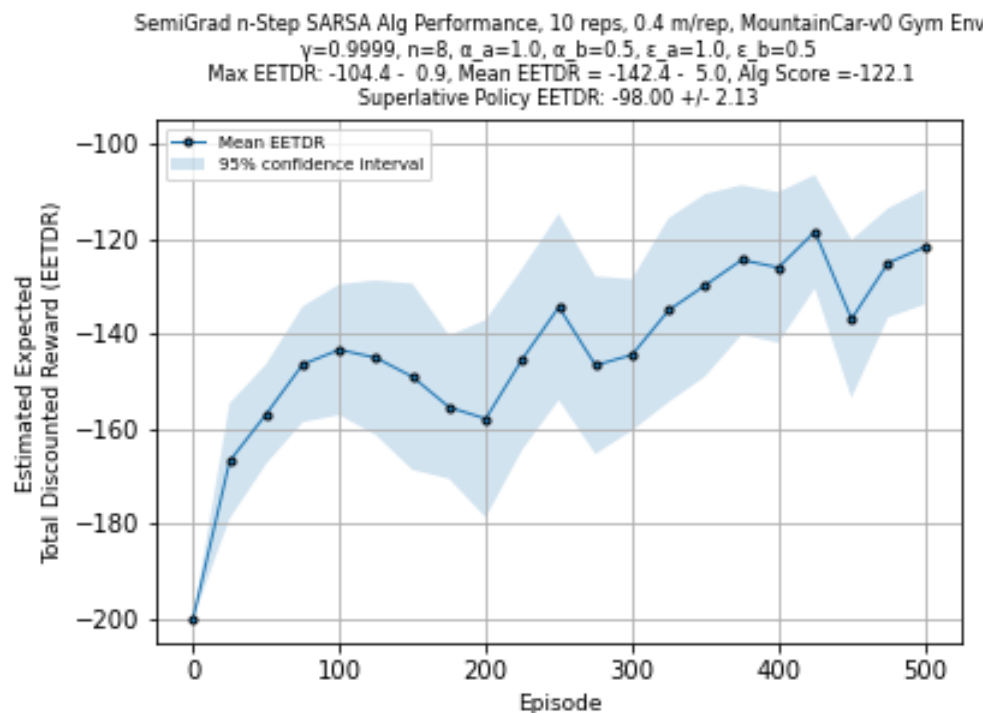


Figure 1: Semi-gradient n -step SARSA learning curve for the Farama Foundation Gymnasium MountainCar-v0 Environment.

```
1 """
2 DSOR 646 - Reinforcement Learning
3 Dr. Matthew Robbins
4 Lesson 15 Handout -- Semi-gradient n-step SARSA with linear VFA
5 Farama Foundation Gymnasium MountainCar-v0 Environment
6 """
7
```

```

8 import gymnasium as gym
9 import numpy as np
10 import time
11 from scipy.stats import t
12 import matplotlib.pyplot as plt
13 from sklearn import metrics
14 import collections
15 from tiles3 import tiles, IHT
16 import copy
17
18
19 # warning suppression if using Python 3.12
20 import warnings
21 warnings.filterwarnings("ignore",category=UserWarning)
22
23 # create Farama OpenAI Gym environment
24 envName = "MountainCar-v0"
25 env = gym.make(envName)
26
27 print(f"\nFarama Foundation Gymnasium version: {gym.__version__}")
28 print(f"Environment: {envName}")
29
30 # Observation and action space
31 print("Observation space: {}".format(env.observation_space))
32 print("Action space: {}".format(env.action_space))
33
34 """ Algorithm Hyperparameters """
35 # algorithm name
36 algName = "SemiGrad n-Step SARSA"
37
38 # VFA tile coding scheme for state-action variables
39 # max size of integer hash table (iht)
40 # too small will impact performance
41 # too large will impact computational time
42 max_size_iht = 2**8 # 8 -- want no collisions, if possible
43 # size parameters for VFA tile coding scheme
44 num_tilings = 4 # partitions (*) 4
45 num_tiles_per_tiling = 4 # elements per partition (*) 4
46
47 # learning rate (smoothing rule)
48 alpha_a = 1.0 # (*) # 1.0
49 alpha_b = 0.5 # (*) # 0.5
50 def alpha(n):
51     return (alpha_a/(1+n)**alpha_b)/num_tilings
52
53 # eps-greedy stepsize rule
54 eps_a = 1.0 # (*) # 1.0
55 eps_b = 0.5 # (*) # 0.5
56 def epsilon(n):
57     return eps_a/(1+n)**eps_b
58
59 # number of steps for computing n-step returns
60 num_steps = 8 # (*) 8
61
62 """ Algorithm hyperparameters """
63
64 # discount factor
65 gamma = 0.9999
66
67 # number of algorithm runs
68 Z = 10
69
70 # number of episodes
71 M = int(0.5e3)

```

```

72
73 # policy evaluation test frequency
74 test_freq = 25
75
76 # number of replications per test
77 num_test_reps = 30
78
79 # seed offset for manual testing
80 offset = 0
81
82 # discrete action space -- number of actions
83 num_actions = env.action_space.n
84
85 # define reference points for use with VFA
86 Slow = np.array(env.observation_space.low)
87 Shigh = np.array(env.observation_space.high)
88 Srange = Shigh-Slow
89 scaleFactor = num_tiles_per_tiling/Srange
90
91 # basis function reports active tiles for (s,a) pair
92 def phi(s,a,iht):
93     return tiles(iht,num_tilings,list(s*scaleFactor),[int(a)])
94
95 # approximate Q-function
96 def Qbar(s,a,w,iht):
97     qhat = 0
98     tiles = phi(s,a,iht)
99     for tile in tiles:
100         qhat += w[tile]
101     return qhat
102
103 # gradient of approximate Q-function
104 def gradQbar(s,a,iht): # returns index numbers of active tiles
105     return phi(s,a,iht)
106
107 # computes action that maximizes performance, as represented by approximate
108 # Q-function, comprised of weight vector w and integer hash table
109 def argmaxQbar(s,w,iht):
110     Qvals = [Qbar(s,a,w,iht) for a in range(num_actions)]
111     return np.argmax(Qvals)
112
113 # function for computing confidence interval
114 def confinterval(data,alpha=0.05):
115     n = np.size(data) # number of data points
116     se = np.std(data,ddof=1)/np.sqrt(n) # standard error
117     ts = t.ppf(1-alpha/2, n-1) # t-score for two-tail test
118     mean = np.mean(data)
119     halfwidth = ts*se
120     return mean, halfwidth
121
122 # define policy evaluation function
123 def evaluate_policy(w,iht, num_reps, seed_mult=1):
124     # initialize test data structure
125     test_data = np.zeros((num_reps))
126     # run num_test_reps replications per test
127     for rep in range(num_reps):
128         # initialize episode complete flags
129         terminated = False
130         truncated = False
131         # initialize episode reward
132         Gtest = 0
133         # initialize the system by resetting the environment, obtain state var
134         state = env.reset(seed=seed_mult*1000+rep)[0]
135         while not(terminated or truncated):

```

```

136         # take action according to current policy, using theta
137         action = argmaxQbar(state,w,iht)
138         # apply action and observe system information
139         state, reward, terminated, truncated, _ = env.step(action)
140         # update episode cumulative reward
141         Gtest += reward
142         test_data[rep] = Gtest
143         mean, hw = confinterval(test_data)
144         return mean, hw
145
146 # initialize the data structure to hold the top 2Z scores and VFA parameters
147 num_best_scores = 2*Z
148 best_scores = [{'ETDR': -np.inf, 'ETDR_hw': np.inf, 'w': None, 'iht': None} for _ in ...
149                 range(num_best_scores)]
150
151 # retain Top 2Z milestone validation testing results for final independent
152 # testing to determine superlative policy across all replications
153 def update_best_scores(mean, hw, w, iht, best_scores):
154     # Find the first score that mean is greater than
155     for i in range(len(best_scores)):
156         if mean > best_scores[i]['ETDR']:
157             # Shift scores and parameters
158             best_scores.insert(i, {'ETDR': np.copy(mean), 'ETDR_hw': np.copy(hw), 'w': ...
159                                   np.copy(w), 'iht': copy.deepcopy(iht)})
160             # We only want the top scores, so remove the last one
161             best_scores.pop()
162             return True
163     return False
164
165 # record each episode's cumulative reward to measure online performance
166 Gzm = []
167 GzmTest = []
168
169 # start timer for executing algorithm design run
170 tic = time.perf_counter()
171
172 # perform each algorithm replication
173 for z in range(Z):
174     # initialize VFA weights for approximate Q-function
175     w = np.zeros((max_size_iht,1))
176     # initialize state-action counter
177     C = np.zeros((max_size_iht,1))
178     # initialize integer hash table
179     iht_VFA = IHT(max_size_iht)
180
181     print(f"\nSemi-grad {num_steps}-step ...
182           SARSA(alpha_a={alpha_a:<3.2f},alpha_b={alpha_b:<3.2f},eps_a={eps_a:<3.2f},eps_b={eps_b:<3.2f}) ...
183           rep {z}...")
184     print(" Policy evaluation milestone validation tests...")
185
186     # loop through each episode
187     for m in range(M):
188         # initialize episode complete flag (when system enters terminal state)
189         terminated = False
190         truncated = False
191         # initialize episode reward
192         Gm = 0
193         # set random number generator seed
194         np.random.seed(int(z*1e6+m+1e5*offset))
195         # initialize the system by resetting the environment, obtain state var
196         state = env.reset(seed=int(z*1e6+m))[0]
197         # initialize state array, record of states
198         state_queue = collections.deque([state])
199         # select action based on epsilon-greedy exploration mechanism

```

```

196 if np.random.rand() > epsilon(m):
197     # act greedy by exploiting currrent knowledge
198     # take best action at current state using current Q
199     action = argmaxQbar(state,w,iht_VFA)
200 else:
201     # act randomly with probability epsilon to explore
202     action = np.random.randint(0,num_actions)
203     # initialize action array, record of actions
204     action_queue = collections.deque([action])
205     # initialize reward array, record of rewards
206     reward_queue = collections.deque([])
207
208 # SARSA main loop (first nml transitions)
209 for _ in range(num_steps-1):
210     # apply action and observe system information
211     next_state, reward, terminated, truncated, _ = env.step(action)
212     # append reward to reward array
213     reward_queue.append(reward)
214     # update episode cumulative reward
215     Gm += reward
216     # append next state to state array
217     state_queue.append(next_state)
218     # select action based on epsilon-greedy exploration mechanism
219     if np.random.rand() > epsilon(m):
220         # act greedy by exploiting currrent knowledge
221         # take best action at current state using Qbar
222         next_action = argmaxQbar(next_state,w,iht_VFA)
223     else:
224         # act randomly with probability epsilon to explore
225         next_action = np.random.randint(0,num_actions)
226     # append next action to action array
227     action_queue.append(next_action)
228
229 # SARSA main loop (episode not complete)
230 while not(terminated or truncated):
231     # apply action and observe system information
232     next_state, reward, terminated, truncated, _ = env.step(action_queue[-1])
233     # append next state to state array
234     state_queue.append(next_state)
235     # append reward to reward array
236     reward_queue.append(reward)
237     # update episode cumulative reward
238     Gm += reward
239     # select action based on epsilon-greedy exploration mechanism
240     if np.random.rand() > epsilon(m):
241         # act greedy by exploiting currrent knowledge
242         # take best action at current state using Qbar
243         next_action = argmaxQbar(next_state,w,iht_VFA)
244     else:
245         # act randomly with probability epsilon to explore
246         next_action = np.random.randint(0,num_actions)
247     # append next action to action array
248     action_queue.append(next_action)
249
250 # temporal difference learning mechanism
251 # compute qhat
252 qhat = np.dot(reward_queue,gamma*np.array(range(len(reward_queue)))) + ...
253         (1-terminated)*gamma*len(reward_queue) * Qbar(next_state,next_action,w,iht_VFA)
254
255 # compute TD error
256 Δ = qhat - Qbar(state_queue[0],action_queue[0],w,iht_VFA)
257
258 # update w vector
259 active_tiles = gradQbar(state_queue[0],action_queue[0],iht_VFA)

```

```

259         C[active_tiles] += 1 # update state-action counter
260         w[active_tiles] += alpha(C[active_tiles])*Δ
261
262         # update state, action, and reward arrays by removing oldest values
263         state_queue.popleft()
264         action_queue.popleft()
265         reward_queue.popleft()
266
267     # SARSA main loop (episode complete)
268     while len(reward_queue)>0:
269         # temporal difference learning mechanism
270         # compute qhat
271         qhat = np.dot(reward_queue, gamma*np.array(range(len(reward_queue))))
272
273         # compute TD error
274         Δ = qhat - Qbar(state_queue[0], action_queue[0], w, iht_VFA)
275
276         # update w vector
277         active_tiles = gradQbar(state_queue[0], action_queue[0], iht_VFA)
278         C[active_tiles] += 1 # update state-action counter
279         w[active_tiles] += alpha(C[active_tiles])*Δ
280
281         # update state, action, and reward arrays by removing oldest values
282         state_queue.popleft()
283         action_queue.popleft()
284         reward_queue.popleft()
285
286         # record performance
287         # print(f"\n    Episode: {m}, Cumulative reward: {Gm}")
288         Gzm.append((z, Gm))
289
290     # test current policy (as represented by current w and iht) every test_freq episodes
291     if m % test_freq == 0:
292         mean, hw = evaluate_policy(w, iht_VFA, num_test_reps)
293         GzmTest.append((z, m, mean, hw))
294
295         # update best scores if necessary
296         if update_best_scores(mean, hw, w, iht_VFA, best_scores):
297             print(f"    Test... Episode: {m:>4}, EETDR CI: {mean:>6.1f} +/- {hw:4.1f} *** ...
298                   New Top {num_best_scores} Policy -- w, iht recorded")
299         else:
300             print(f"    Test... Episode: {m:>4}, EETDR CI: {mean:>6.1f} +/- {hw:4.1f}")
301
302     # last test of current algorithm run
303     mean, hw = evaluate_policy(w, iht_VFA, num_test_reps)
304     GzmTest.append((z, M, mean, hw))
305
306     # update best scores if necessary
307     if update_best_scores(mean, hw, w, iht_VFA, best_scores):
308         print(f"    Test... Episode: {M:>4}, EETDR CI: {mean:>6.1f} +/- {hw:4.1f} *** New Top ...
309               {num_best_scores} Policy -- w, iht recorded")
310     else:
311         print(f"    Test... Episode: {M:>4}, EETDR CI: {mean:>6.1f} +/- {hw:4.1f}\n    ...
312               *-----* Current Top 5 EETDRs: {best_scores[0]['ETDR']:>6.1f}, ...
313               {best_scores[1]['ETDR']:>6.1f}, {best_scores[2]['ETDR']:>6.1f}, ...
314               {best_scores[3]['ETDR']:>6.1f}, {best_scores[4]['ETDR']:>6.1f}")
315
316     toc = time.perf_counter()
317     print(f"\nExecuted {Z} algorithm reps in {toc - tic:0.4f} seconds.")
318
319     # Identify Superlative Policy via final independent testing of Top {num_best_scores} ...
320     # policies from milestone validation testing...")
321
322     # initialize list of means and half-widths for testing top policies

```

```

317 mean_values = []
318 hw_values = []
319
320 # loop through top policies stored in best_scores to find superlative policy
321 # via final testing with different random number seed and more reps
322 for i, score in enumerate(best_scores):
323     mean, hw = evaluate_policy(score['w'], score['iht'], 2*num_test_reps,2)
324     print(f"    Rank: {i+1:>2d} policy final test... EETDR CI: {mean:>6.2f} +/- {hw:>4.2f}")
325     mean_values.append(mean)
326     hw_values.append(hw)
327
328 # determine superlative policy and record its mean and half-width
329 indSupETDR = np.argmax(np.array(mean_values))
330 supETDR = mean_values[indSupETDR]
331 supETDRhw = hw_values[indSupETDR]
332
333 # VFA parameter-values representing superlative policy for algorithm design run
334 w = np.copy(best_scores[indSupETDR]['w'])
335 iht_VFA = copy.deepcopy(best_scores[indSupETDR]['iht'])
336
337 """
338 plot algorithm performance results
339 """
340
341 # organize validation milestone testing data for displaying learning curve
342 npGzmTest = np.array(GzmTest)
343 TestEETDR = np.reshape(npGzmTest[:,2], (Z,int(npGzmTest.shape[0]/Z)))
344
345 # policy evaluation via milestone testing (validation) -- Learning Curve
346 avgTestEETDR = np.mean(TestEETDR,axis=0) # mean at each milestone
347 avgTestSE = np.std(TestEETDR,axis=0)/np.sqrt(Z)
348 avgTestHW = t.ppf(1-0.05/2, Z-1)*avgTestSE # half-width at each milestone
349
350 # measure peak algorithm performance across multiple replications
351 maxTestEETDR = np.max(TestEETDR,axis=1)
352 meanMaxTestEETDR = np.mean(maxTestEETDR)
353 maxTestSE = np.std(maxTestEETDR,ddof=1)/np.sqrt(Z)
354 maxTestME = t.ppf(0.95, Z-1)*maxTestSE
355
356 # measure average algorithm performance across multiple replications
357 AULC = [metrics.auc(np.arange(0,M+1,test_freq), TestEETDR[z,:])/M for z in range(Z)]
358 meanAULC = np.round(np.mean(AULC),1)
359 meAULC = t.ppf(0.95, Z-1)*np.std(AULC,ddof=1)/np.sqrt(Z)
360
361 # algorithm score values hyperparameters resulting in both peak and average performance
362 # peak performance valued slightly more than average performance
363 # reliability valued by using lower bound of single-sided confidence interval, incorporating ...
364     margin of error (ME)
365 AlgScore = 0.6*(meanMaxTestEETDR-maxTestME) + 0.4*(meanAULC-meAULC)
366
367 # create and show learning curve figure
368 plt.figure(0)
369 plt.plot(np.arange(0,M+1,test_freq),avgTestEETDR, marker = 'o', ms = 3, mec = 'k', linewidth ...
370     = 1, label='Mean EETDR')
371 plt.fill_between(
372     np.arange(0,M+1,test_freq),
373     avgTestEETDR-avgTestHW,
374     avgTestEETDR+avgTestHW,
375     alpha=0.2,
376     label='95% confidence interval')
377 plt.xlabel('Episode',fontSize=9)
378 plt.ylabel('Estimated Expected Total Discounted Reward (EETDR)',fontSize=9)
379 plt.title(f'{algName} Alg Performance, {Z} reps, {np.round((toc-tic)/(Z)/60,1)} m/rep, ...
380     {envName} Gym Env\nn={num_steps}, gamma={gamma}, alpha_a={alpha_a}, alpha_b={alpha_b}, ...

```

```

        eps_a={eps_a}, eps_b={eps_b}\n Max EETDR: {meanMaxTestEETDR:>6.1f} - {maxTestME:4.1f}, ...
        Mean EETDR = {meanAULC:>6.1f} - {meAULC:4.1f}, Alg Score ={AlgScore:6.1f}\nSuperlative ...
        Policy EETDR: {supEETDR:>6.2f} +/- {supEETDRhw:4.2f}',fontsize=8)
378 plt.legend(loc="upper left",fontsize=7)
379 plt.xlim([-0.05*M, 1.05*M])
380 plt.ylim([-205, -95])
381 plt.grid(which='both')
382 plt.show()

```